Lecture 07:

Introduction to Artificial Neural Networks (ANN)



Markus Hohle

University of California, Berkeley

Machine Learning Algorithms

MSSE 277B, 3 Units

Course Map

Lecture 1: Course Overview and Introduction to Machine Learning

Lecture 2: Bayesian Methods in Machine Learning

classic ML tools & algorithms

Lecture 3: Dimensionality Reduction: Principal Component Analysis

Lecture 4: Linear and Non-linear Regression and Classification

Lecture 5: Unsupervised Learning: K-Means, GMM, Trees

Lecture 6: Adaptive Learning and Gradient Descent Optimization Algorithms

Lecture 7: Introduction to Artificial Neural Networks - The Perceptron

ANNs/AI/Deep Learning

Lecture 8: Introduction to Artificial Neural Networks - Building Multiple Dense Layers

Lecture 9: Convolutional Neural Networks (CNNs) - Part I

Lecture 10: CNNs - Part II

Lecture 11: Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTMs)

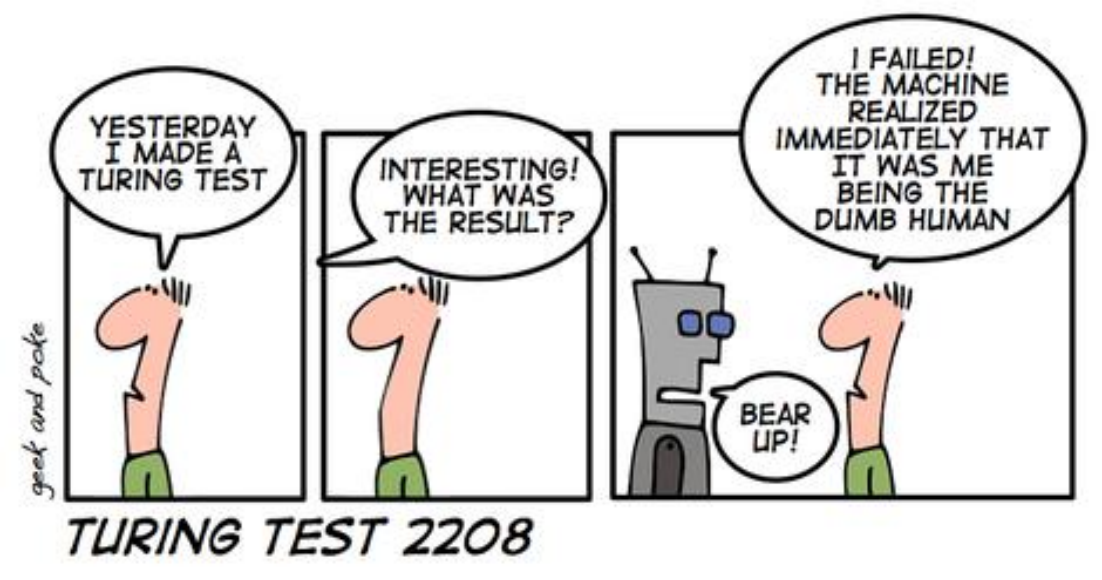
Lecture 12: Combining LSTMs and CNNs

Lecture 13: Running Models on GPUs and Parallel Processing

Lecture 14: Project Presentations

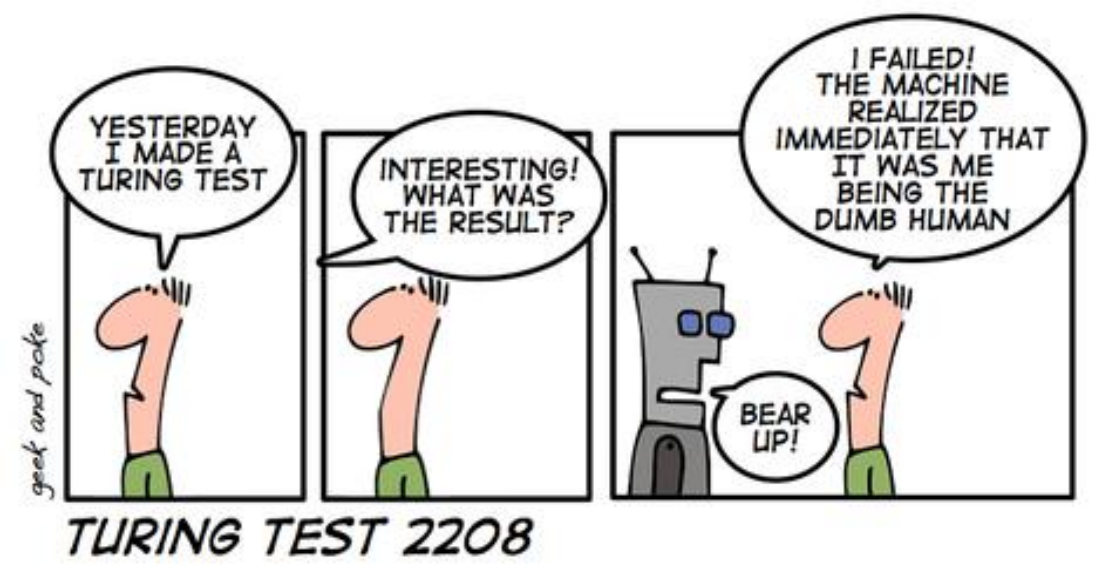
Lecture 15: Transformer

Lecture 16: GNN



Outline

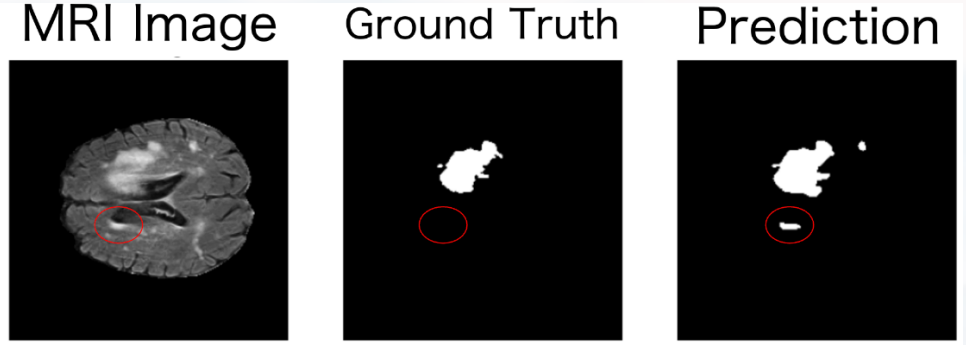
- **Motivation & Overview**
- **The Perceptron aka Neuron**
- **Building a Perceptron**



Outline

- **Motivation & Overview**
- The Perceptron aka Neuron
- Building a Perceptron

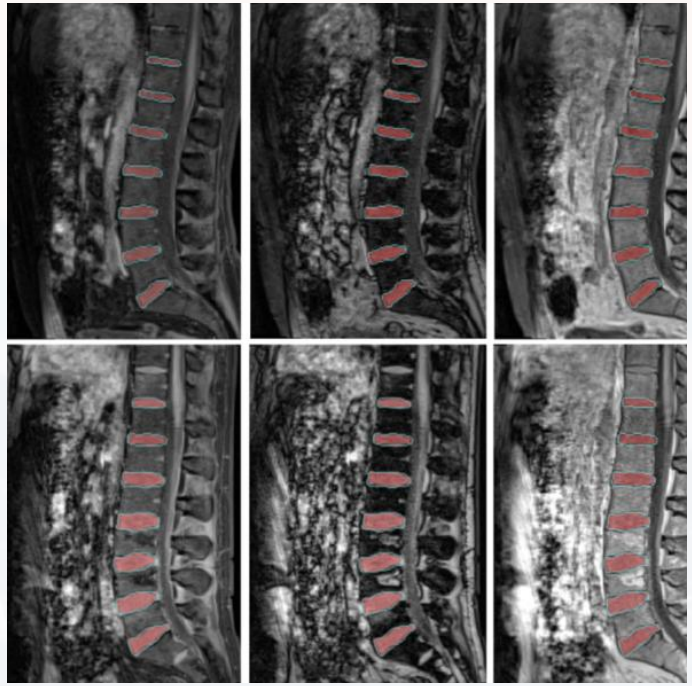
Why Artificial Neural Networks?

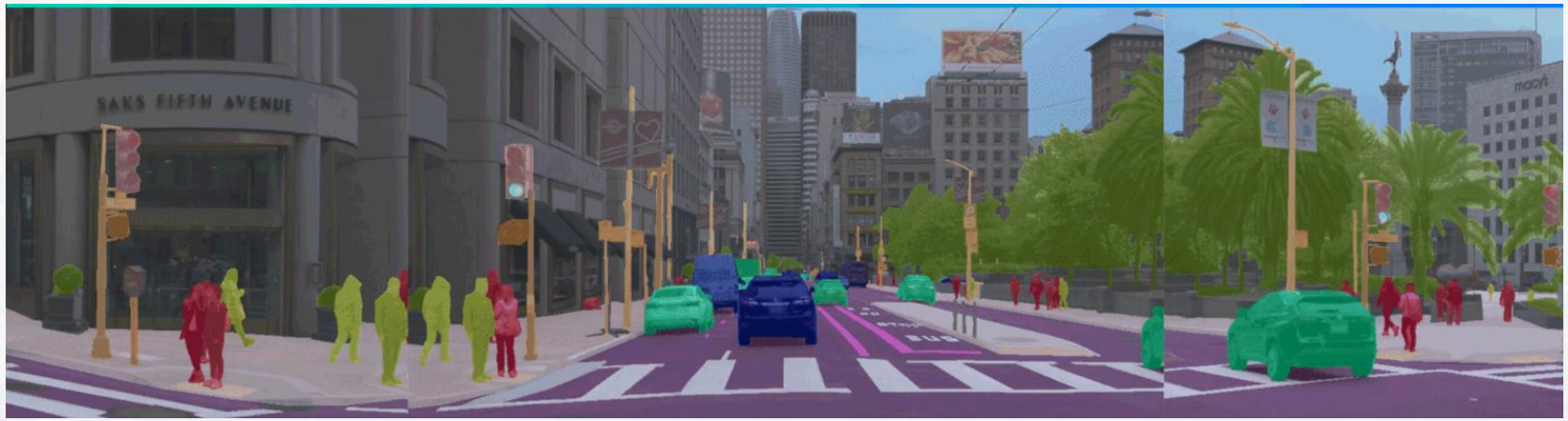


MINT: Modality Independent Neuroimaging Transformer

Jade Warren[1], Manula Dombagahawatta[1], Noah May[1], Shannon Jackson[1], Carlos Lopez[1] and Brendan Dang[1]

277B final project, Spring 2026

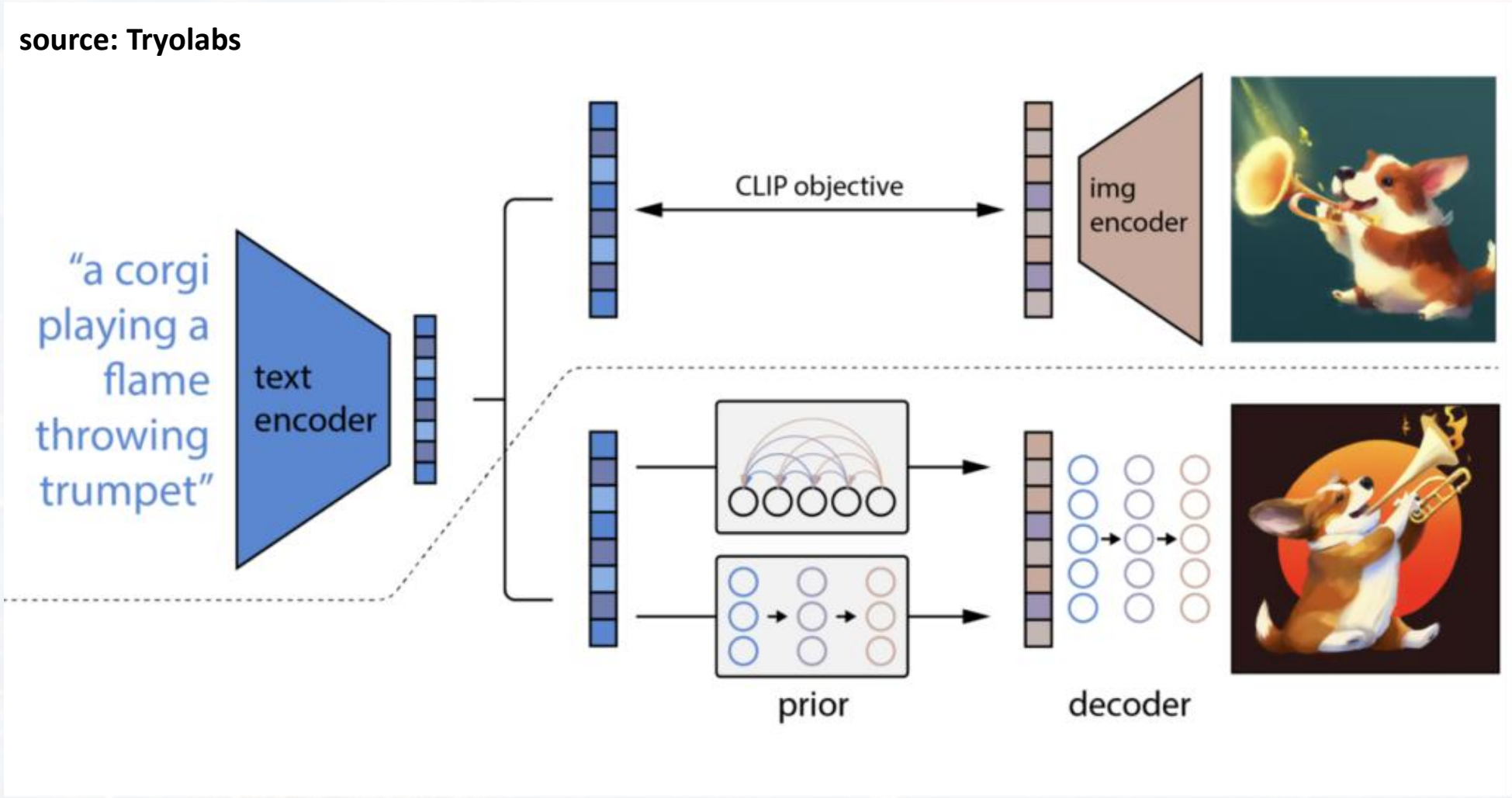




source: Waymo

Why Artificial Neural Networks?

source: Tryolabs



Why Artificial Neural Networks?

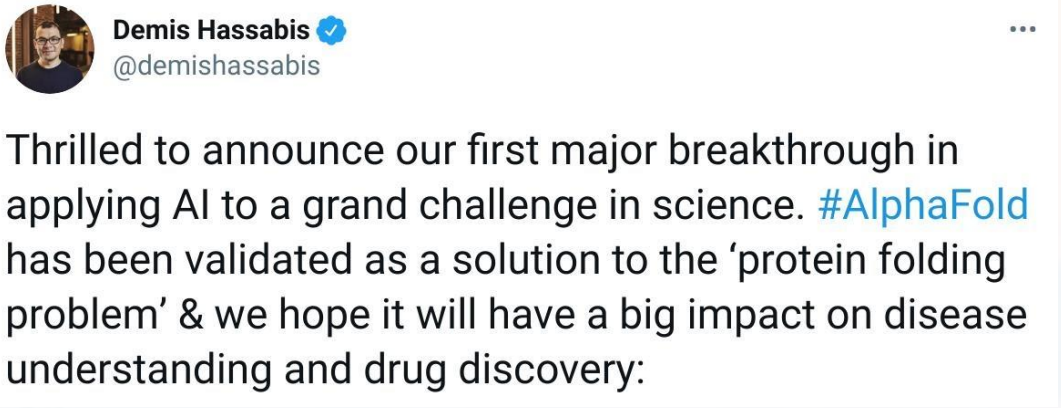
Article

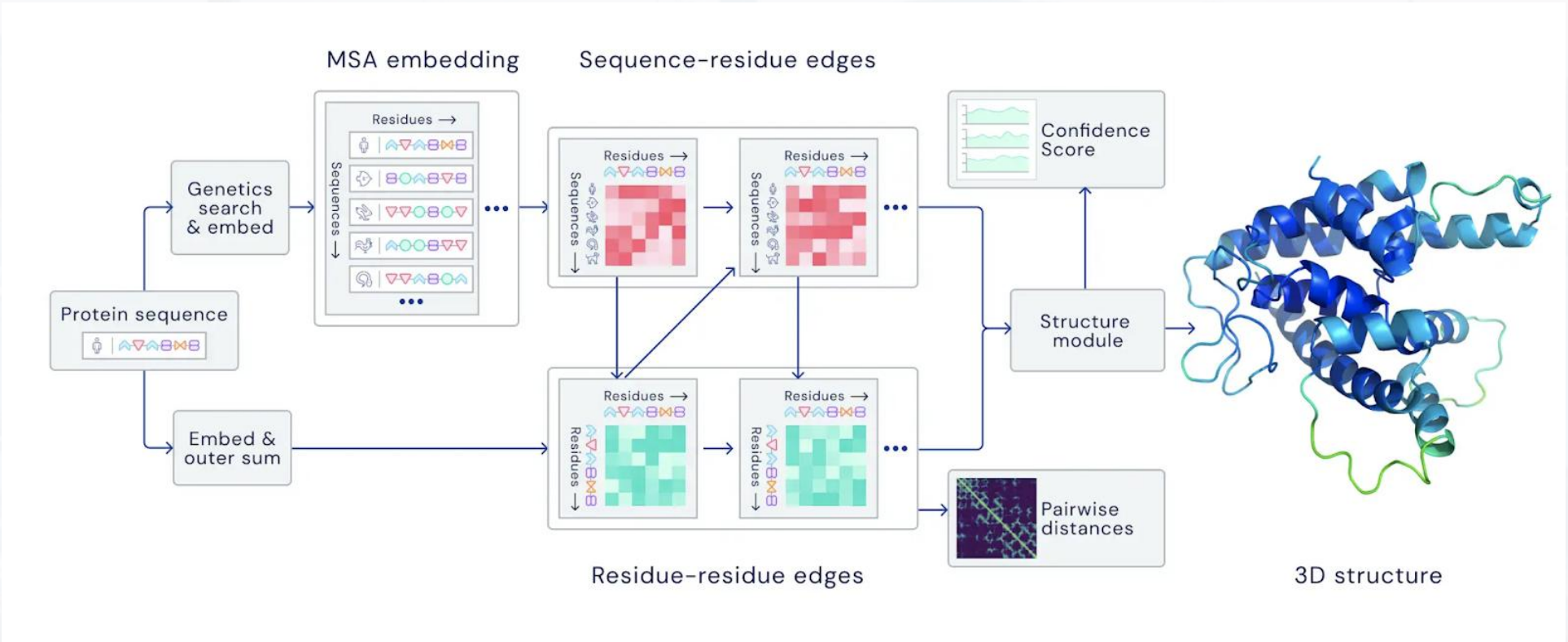
DeepSOCIAL: Social Distancing Monitoring and Infection Risk Assessment in COVID-19 Pandemic

YOLOv4-based *Deep Neural Network* (DNN) model for automated people detection in the crowd in indoor and outdoor environments using common CCTV security cameras. The proposed DNN model in combination with an adapted inverse perspective mapping (IPM) technique and SORT tracking algorithm leads to a robust people detection and social distancing monitoring. The model has been trained against two most comprehensive datasets by the time of the research—the Microsoft Common Objects in Context (MS COCO) and Google Open Image datasets. The system has been

health authorities have set the 2-m physical distancing as a mandatory safety measure in shopping centres, schools and other covered areas. In this research, we develop a hybrid *Computer Vision* and YOLOv4-based *Deep Neural Network* (DNN) model for automated people detection in the crowd in indoor and outdoor environments using common CCTV security cameras. The proposed DNN model in combination with an adapted inverse perspective mapping (IPM) technique and SORT tracking algorithm leads to a robust people detection and social distancing monitoring. The model has been trained against two most comprehensive datasets by the time of the research—the Microsoft Common Objects in Context (MS COCO) and Google Open Image datasets. The system has been evaluated against the Oxford Town Centre dataset (including 150,000 instances of people detection)

Why Artificial Neural Networks?

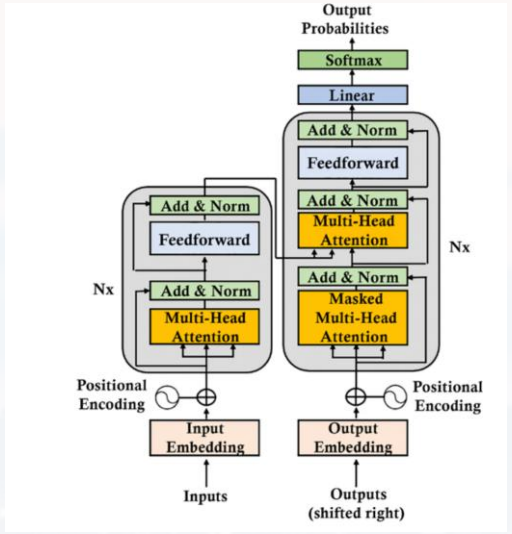




ANN - Intro:

Why Artificial Neural Networks?

ChatGPT



language models
“understanding” context



AI art



AI generated videos



Why Artificial Neural Networks?

Reuters

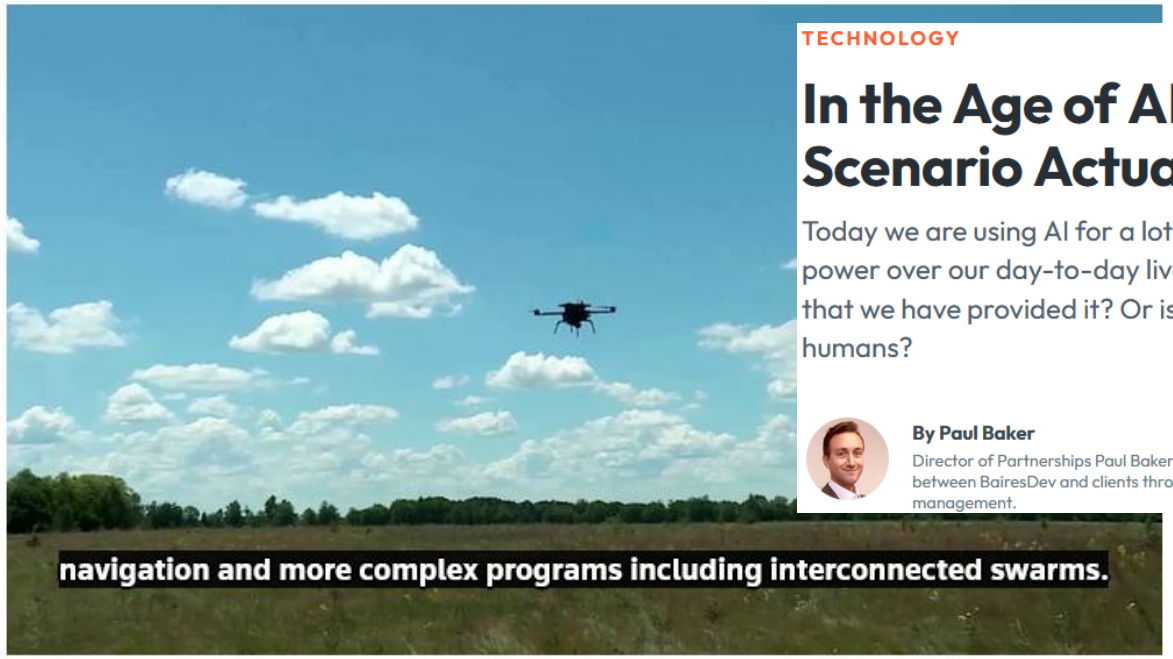
World | US Election | Business | Markets | Sustainability | Legal | Breakingviews | Technology

Artificial Intelligence | Ukraine and Russia at War

Ukraine rushes to create AI-enabled war drones

By Max Hunder

July 18, 2024 3:36 PM GMT+2 · Updated 2 months ago



TECHNOLOGY

In the Age of AI, Can a Terminator Scenario Actually Happen?

Today we are using AI for a lot of reasons. And unknowingly, we are giving it a lot of power over our day-to-day lives. But can AI turn upon us with the same resources that we have provided it? Or is it a fantasy to think that AI can ever go against humans?

By Paul Baker

Director of Partnerships Paul Baker builds strong business relationships between BairesDev and clients through strategy and partnership management.

6 min read Share

In the next lectures we will learn about:

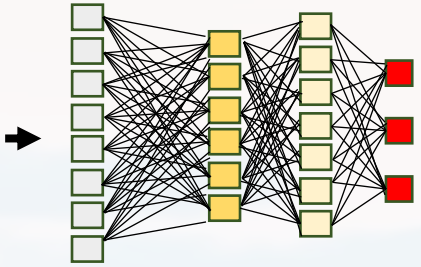
- classical multi-layer ANN	regression, classification
- **C**onvolution **N**eural **N**etworks	image classification, segmentation, sequence analysis
- **R**ecurrent **N**eural **N**etworks	time series forecasting and classification
- **L**ong-**S**hort-**T**erm-**M**emory ANNs	time series forecasting and classification, language processing
- combining the above networks	
- **G**raph **N**eural **N**etworks	molecular 3D structures, process/workflow planning
- Transformers	time series analysis, **L**arge **L**anguage **M**odels, **N**atural **L**anguage **P**rocessing, hybrid architectures

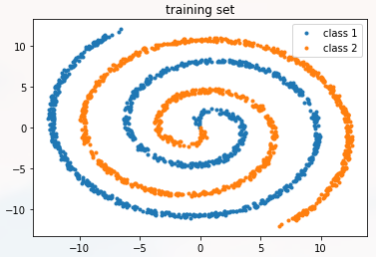
bonus: - running ANNs on GPUs using Cuda in PyTorch → 20 – 50 times faster!
- Parallel Processing in Python

ANN - Intro:

from scratch!

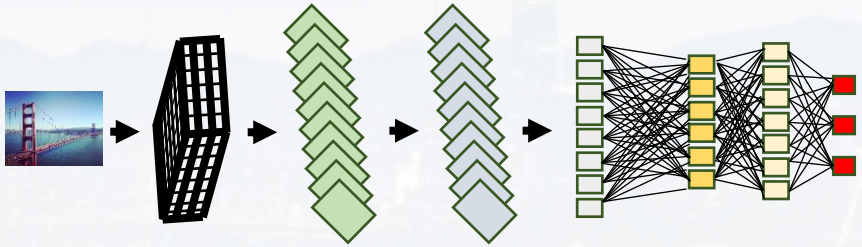
fully connected
(ANN)





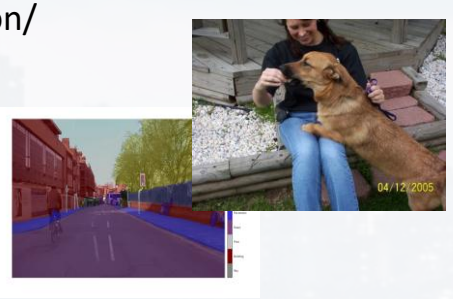
classification/
regression

convolution
(CNN)

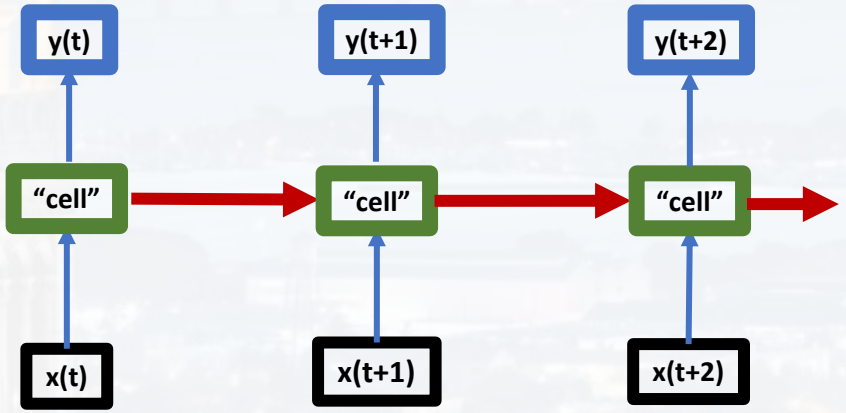


classification/
regression



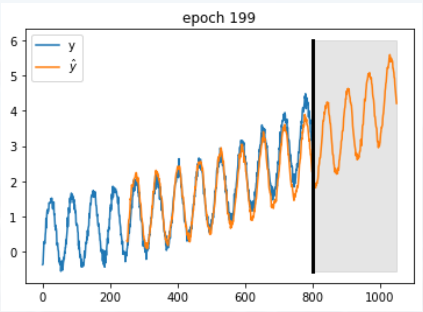


recurrent
(RNN, LSTM)

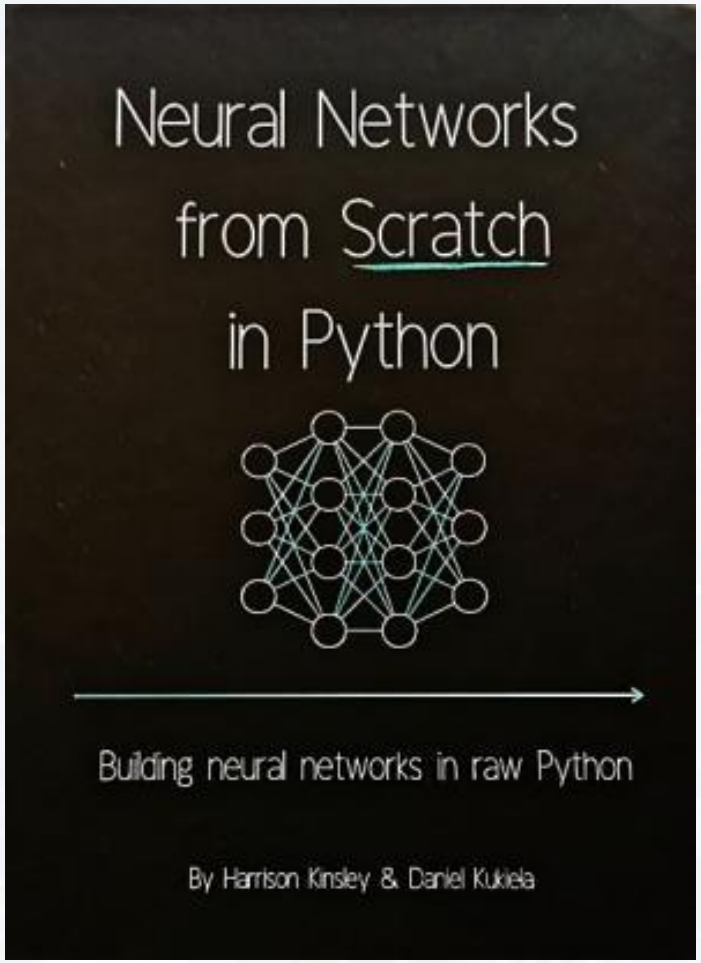


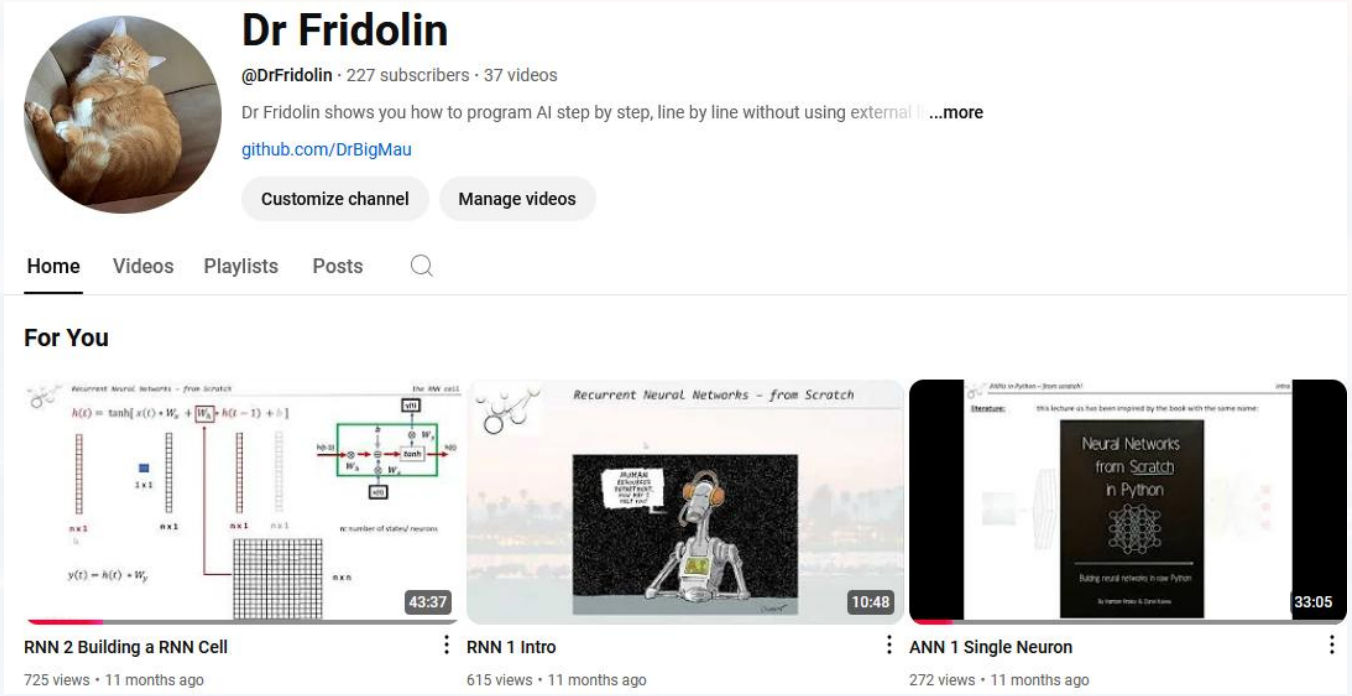
...

classification/
Regression, **prediction**



Tutorials and Books





building ANNs, RNNs, LSTMs, CNNs **from scratch**
(only `numpy` and `matplotlib`)

Tutorials and Books

examples and **application** (CNN/RNN/LSTM and more):



Jason Brownlee
Machine Learning Mastery

all about **transformers, LLM/NLP and way more**



Andrej Karpathy

@AndrejKarpathy · 451K subscribers · 14 videos

FAQ ...more

karpathy.ai and 2 more links

Subscribe



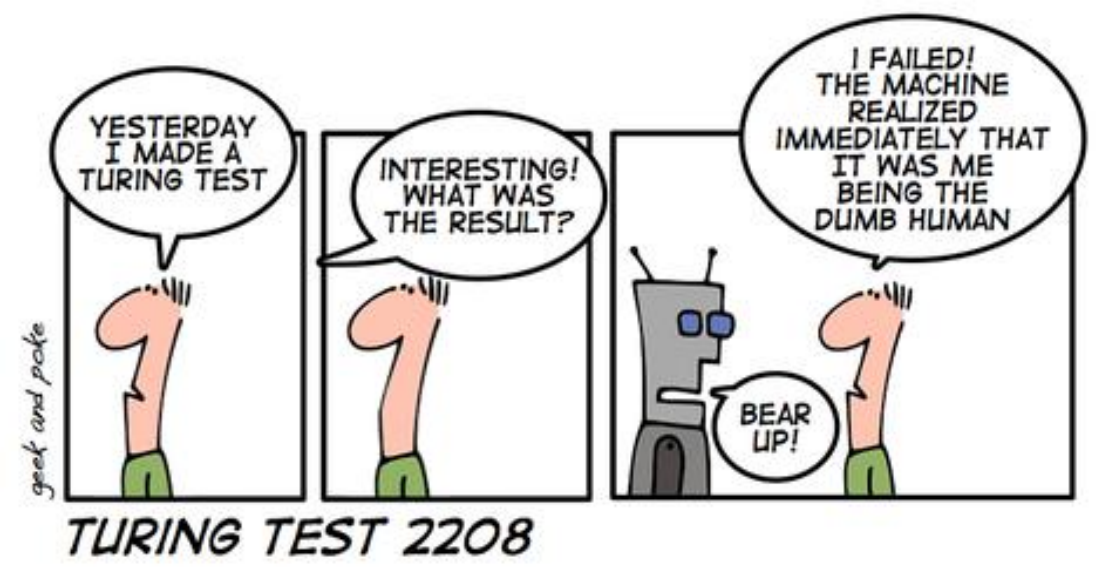
Mısra Turp

@misraturp · 38.2K subscribers · 163 videos

Here is where we learn! This is a space to take it slow

misraturp.com/roadmap and 3 more links

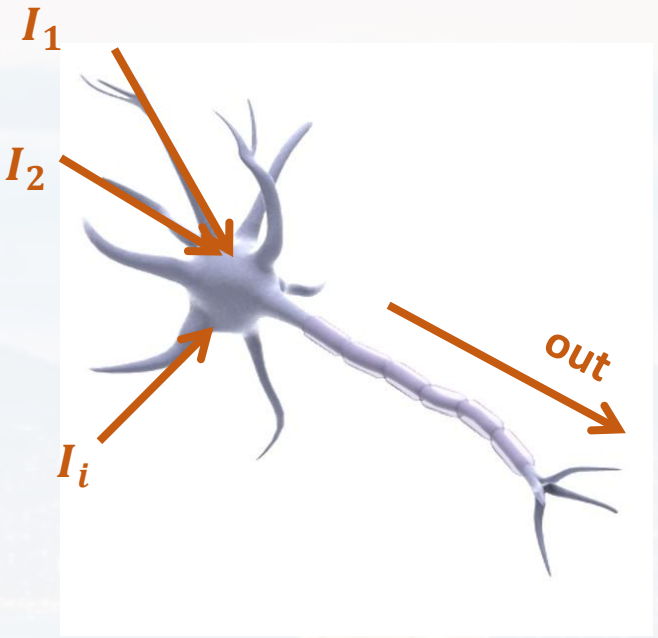
Subscribe



Outline

- Motivation & Overview
- **The Perceptron aka Neuron**
- Building a Perceptron

the perceptron:



what we know…

- inputs enter the neuron
- something happens inside the neuron
- neuron generates an output

…how we could model it

- inputs must be weighted (important vs unimportant input)
- **learning process: changing weights**
- output = sum of weighted inputs

I_i input *i*

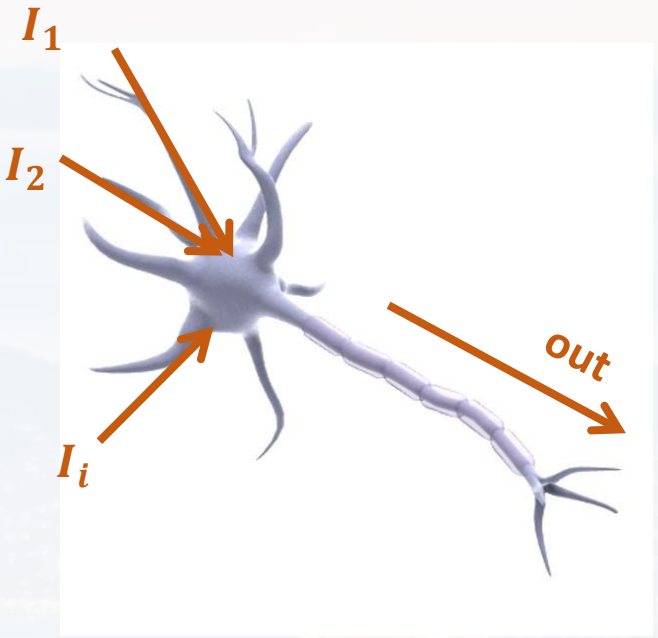
w_i corresponding weight

b bias (base potential)

$$net = \sum_i I_i \cdot w_i + b$$

dot product

the perceptron:



$$net = \sum_i I_i \cdot w_i + b$$

dot product

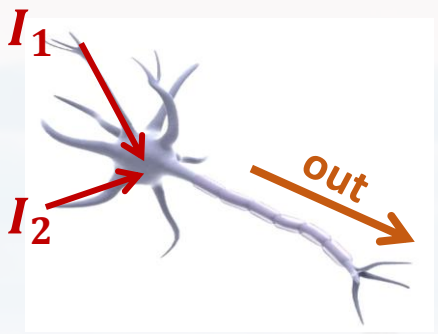
I_i input *i*
w_i corresponding weight
b bias (base potential)

recall: linear models

$$y_k = \beta_0 + \sum_{n=1}^{N} \beta_n x_{k,n} + \varepsilon_k$$

$$\begin{pmatrix} y_1 \\ \dots \\ y_k \\ \dots \\ y_K \end{pmatrix} = \begin{pmatrix} 1 & x_{11} & x_{12} \dots x_{1n} \dots x_{1N} \\ \dots & \dots & \dots \\ 1 & x_{k1} & x_{kn} \\ 1 & \dots & \dots \\ 1 & x_{K1} & x_{K2} \dots x_{Kn} \dots x_{KN} \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \dots \\ \beta_n \\ \dots \\ \beta_N \end{pmatrix} + \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \dots \\ \varepsilon_k \\ \dots \\ \varepsilon_K \end{pmatrix}$$

the perceptron:



$$net = \sum_i I_i \cdot w_i + b$$

I_i input *i*
w_i corresponding weight
b bias (base potential)

dot product

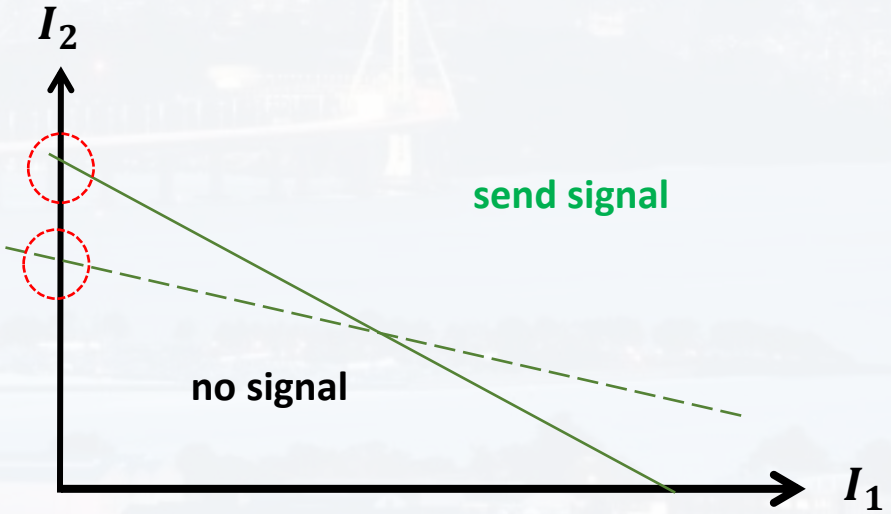
simple example:

one neuron with a **switch**, threshold **T** and **two** input channels

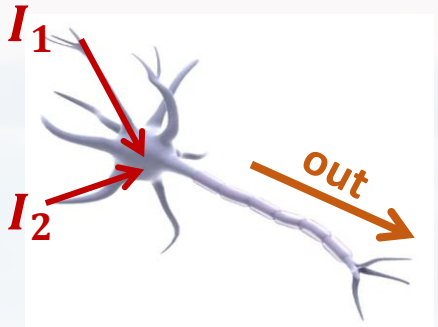
fire if: $b + I_1 w_1 + I_2 w_2 > T$

$$I_2 = -\frac{w_1}{w_2} I_1 + \frac{T - b}{w_2}$$

slope **offset**



the perceptron:

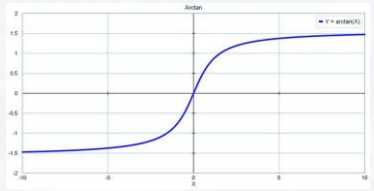


$$net = \sum_i I_i \cdot w_i + b$$

I_i input *i*
w_i corresponding weight
b bias (base potential)

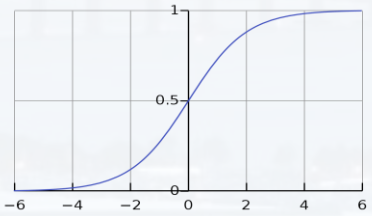
net → some activation function ***f*** → actual output

$y = \arctan(net)$



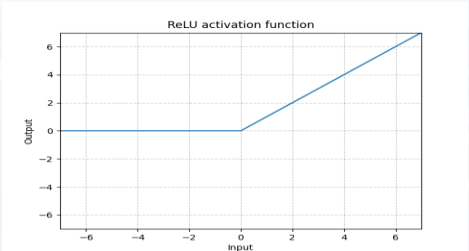
$(-\infty;\ +\infty) \rightarrow (-\pi/2;\ +\pi/2)$

$y = \sigma(net)$



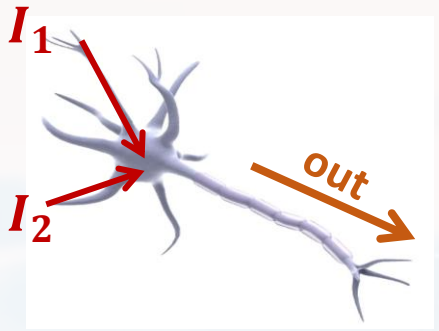
$(-\infty;\ +\infty) \rightarrow (0; 1)$

$y = ReLU(net)$
$= \max(0, net)$



$(-\infty;\ +\infty) \rightarrow (0; +\infty)$

the perceptron:



$$net = \sum_i I_i \cdot w_i + b$$

I_i input *i*
w_i corresponding weight
b bias (base potential)
f activation function

$$y = f(net)$$

learning:

$$net = \sum_i I_i \cdot w_i + b \xrightarrow{\text{activation function } f} y = f(net) \xrightarrow{\text{objective function } E} E = \frac{1}{2}(t - y)^2$$

target output t

finding best w_i by **minimizing** *E*

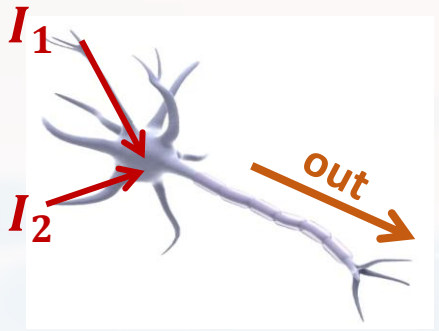
$$\Delta w_i = w_i(new) - w_i(old) = -\alpha \frac{\partial E}{\partial w_i}$$

gradient descent with learning rate α

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial y}\frac{\partial y}{\partial net}\frac{\partial net}{\partial w_i}$$

chain rule

the perceptron:



$$net = \sum_i I_i \cdot w_i + b$$

I_i input *i*
w_i corresponding weight
b bias (base potential)
f activation function

$$y = f(net)$$

learning:

activation function ***f*** objective function ***E***

$$net = \sum_i I_i \cdot w_i + b \longrightarrow y = f(net) \longrightarrow E = \frac{1}{2}(t - y)^2$$

target output ***t***

finding best w_i by **minimizing** *E*

$$\Delta \boldsymbol{w_i} = \boldsymbol{w_i}(\boldsymbol{new}) - \boldsymbol{w_i}(\boldsymbol{old}) = -\boldsymbol{\alpha}\frac{\boldsymbol{\partial E}}{\boldsymbol{\partial w_i}}$$

gradient descent with learning rate $\boldsymbol{\alpha}$

$$\frac{\boldsymbol{\partial E}}{\boldsymbol{\partial w_i}} = \frac{\boldsymbol{\partial E}}{\boldsymbol{\partial y}}\frac{\boldsymbol{\partial y}}{\boldsymbol{\partial net}}\frac{\boldsymbol{\partial net}}{\boldsymbol{\partial w_i}} = -(\boldsymbol{t} - \boldsymbol{y})\, \boldsymbol{f'}(\boldsymbol{net})\, \boldsymbol{I_i}$$

learning:

activation function ***f*** objective function ***E***

$$net = \sum_i I_i \cdot w_i + b \longrightarrow y = f(net) \longrightarrow E = \frac{1}{2}(t - y)^2$$

target output ***t***

finding best w_i by **minimizing** *E*

$$\Delta \boldsymbol{w_i} = \boldsymbol{w_i}(\boldsymbol{new}) - \boldsymbol{w_i}(\boldsymbol{old}) = -\boldsymbol{\alpha}\frac{\boldsymbol{\partial E}}{\boldsymbol{\partial w_i}}$$

gradient descent with learning rate $\boldsymbol{\alpha}$

$$\frac{\boldsymbol{\partial E}}{\boldsymbol{\partial w_i}} = \frac{\boldsymbol{\partial E}}{\boldsymbol{\partial y}}\frac{\boldsymbol{\partial y}}{\boldsymbol{\partial net}}\frac{\boldsymbol{\partial net}}{\boldsymbol{\partial w_i}} = \boxed{-(\boldsymbol{t} - \boldsymbol{y})\ \boldsymbol{f'(net)}}\ \boxed{\boldsymbol{I_i}}$$

inner derivative

outer derivatives

$\boldsymbol{f'(net)}$ depends on the activation function

The required change of ***E*** propagates back to the changes of $\boldsymbol{w_i}$ → **backpropagation**

learning:

$$net = \sum_i I_i \cdot w_i + b \longrightarrow y = f(net) \longrightarrow E = \frac{1}{2}(t - y)^2$$

activation function ***f*** objective function ***E***

target output ***t***

finding best w_i by **minimizing** *E*

$$\Delta \boldsymbol{w_i} = \boldsymbol{w_i}(\boldsymbol{new}) - \boldsymbol{w_i}(\boldsymbol{old}) = -\boldsymbol{\alpha}\frac{\boldsymbol{\partial E}}{\boldsymbol{\partial w_i}}$$

gradient descent with learning rate $\boldsymbol{\alpha}$

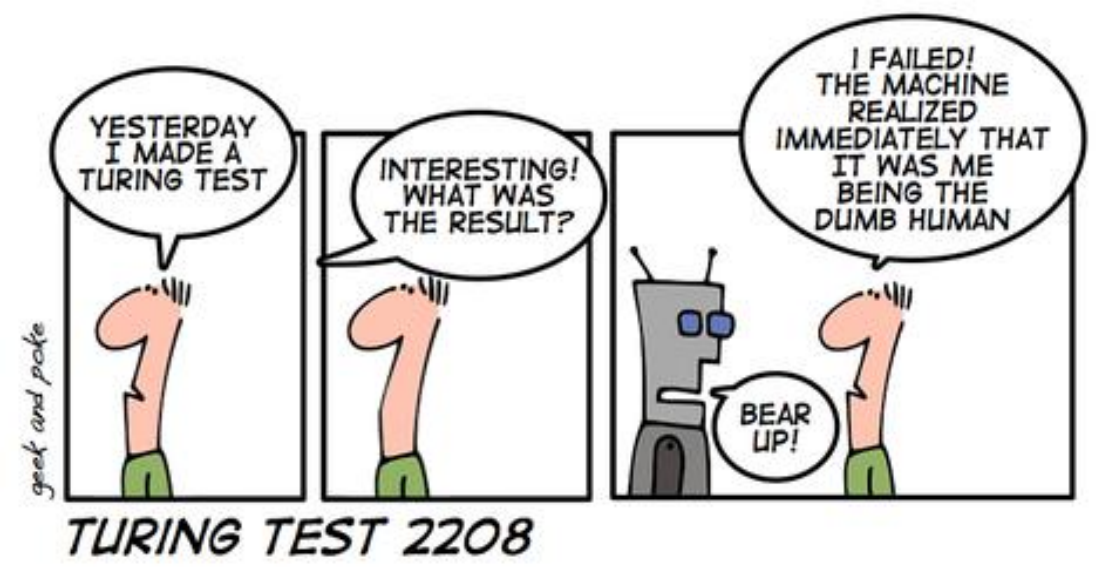
$$\frac{\boldsymbol{\partial E}}{\boldsymbol{\partial w_i}} = \frac{\boldsymbol{\partial E}}{\boldsymbol{\partial y}}\frac{\boldsymbol{\partial y}}{\boldsymbol{\partial net}}\frac{\boldsymbol{\partial net}}{\boldsymbol{\partial w_i}} = \boxed{-(\boldsymbol{t}-\boldsymbol{y})\ \boldsymbol{f'}(\boldsymbol{net})}\ \boxed{\boldsymbol{I_i}}$$

inner derivative outer derivatives

The required change of ***E*** propagates back to the changes of $\boldsymbol{w_i}$ → **backpropagation**

$$\Delta \boldsymbol{w_i} = \boldsymbol{w_i}(\boldsymbol{new}) - \boldsymbol{w_i}(\boldsymbol{old}) = \boldsymbol{\alpha}\ (\boldsymbol{t}-\boldsymbol{y})\ \boldsymbol{f'}(\boldsymbol{net})\ \boldsymbol{I_i}$$

$$\Delta \boldsymbol{b} = \boldsymbol{b}(\boldsymbol{new}) - \boldsymbol{b}(\boldsymbol{old}) \quad = \boldsymbol{\alpha}\ (\boldsymbol{t}-\boldsymbol{y})\ \boldsymbol{f'}(\boldsymbol{net})\ \cdot \boldsymbol{1}$$

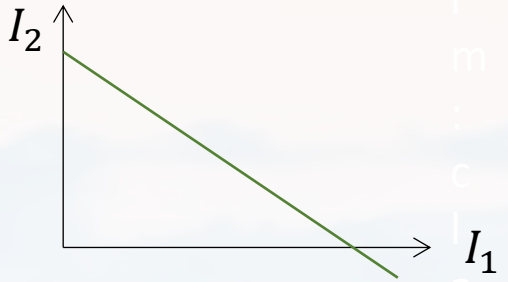


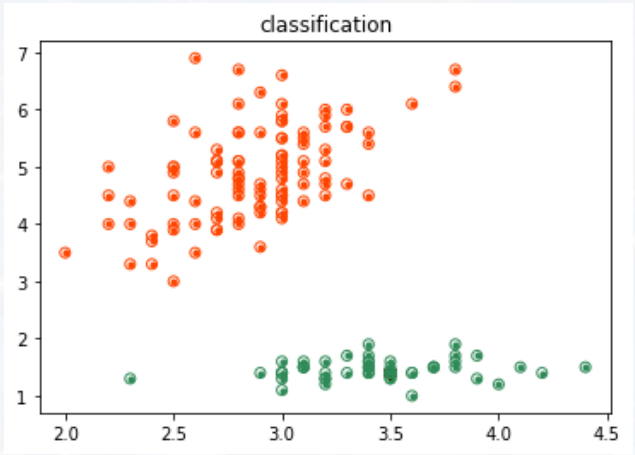
Outline

- Motivation & Overview
- The Perceptron aka Neuron
- **Building a Perceptron**

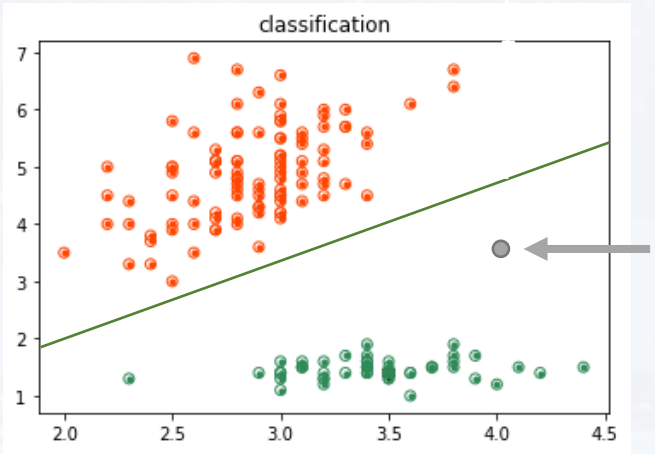
Building a Perceptron

one neuron
two features
two classes
N data points





goal



classifying a new data point

see the Jupyter Notebook `Perceptron.ipynb` for details

main part of the code:

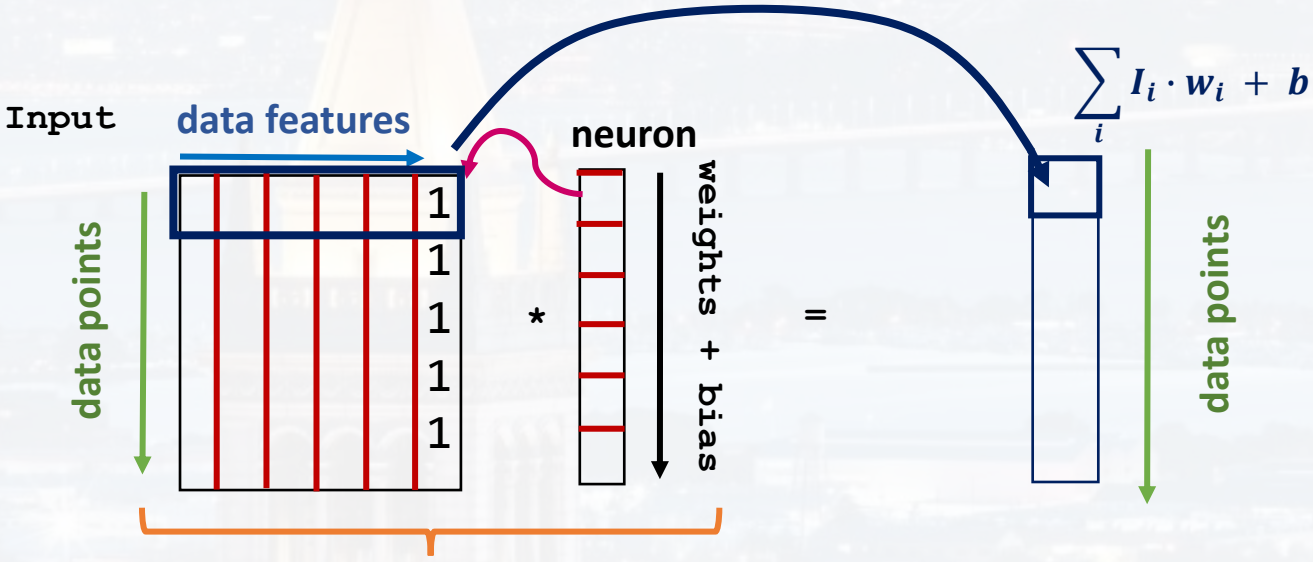
$$net = \sum_i I_i \cdot w_i + b$$

dot product

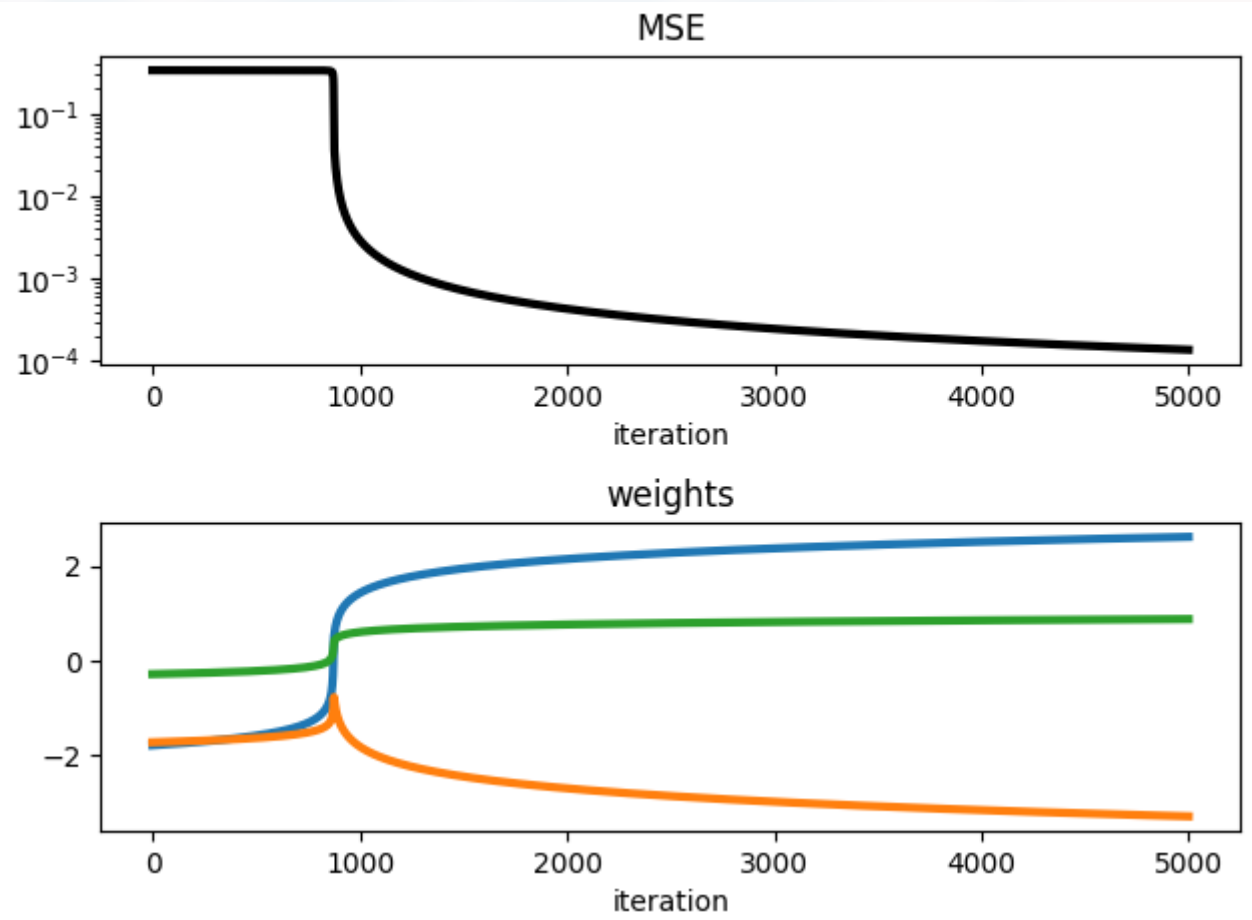
data features

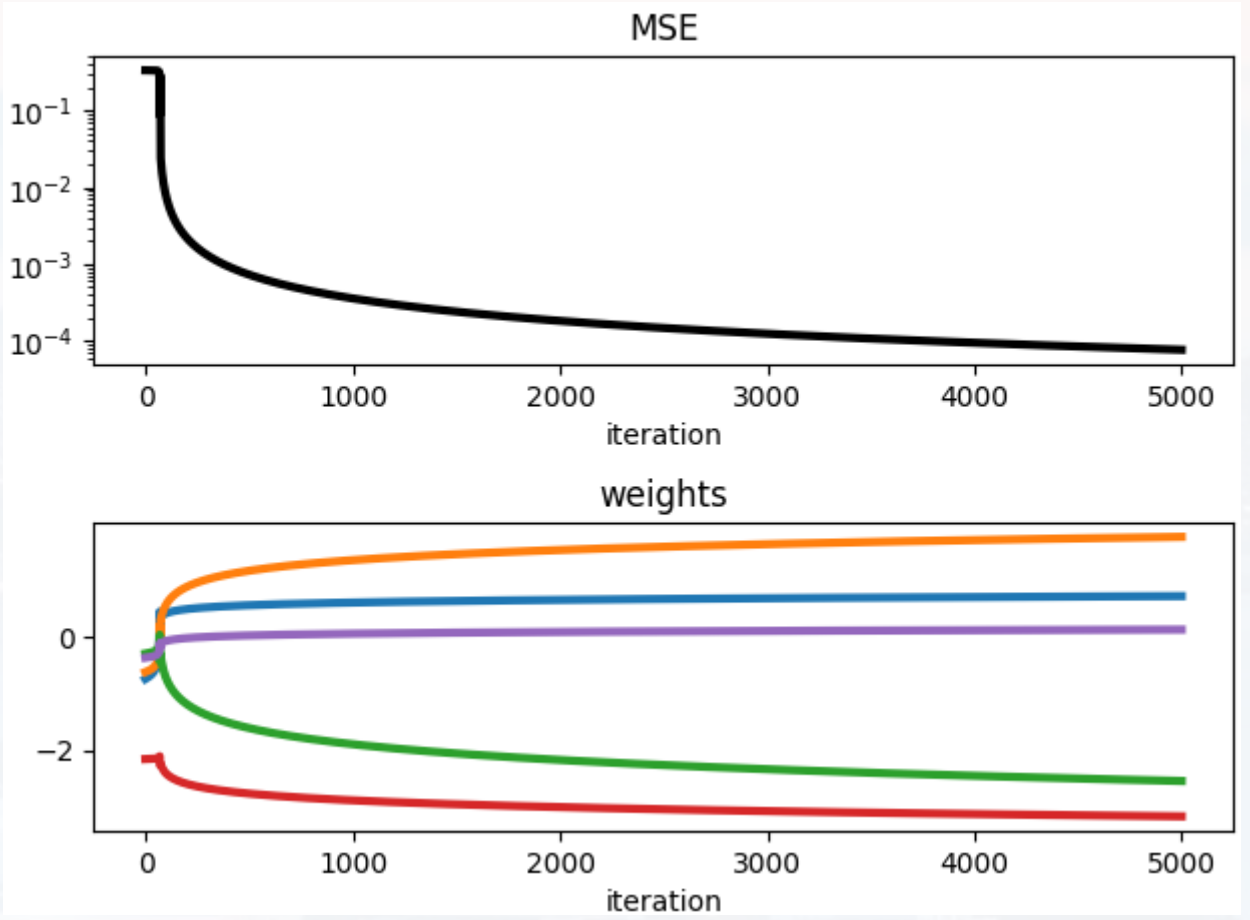
data points

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]]
```

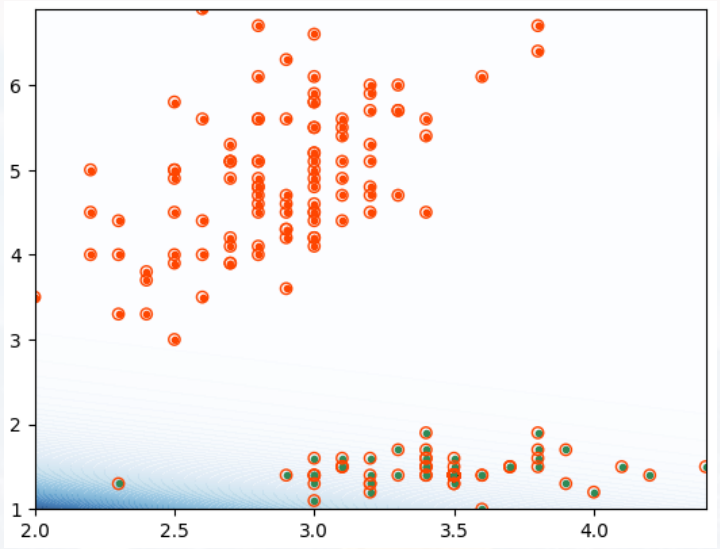


The training process

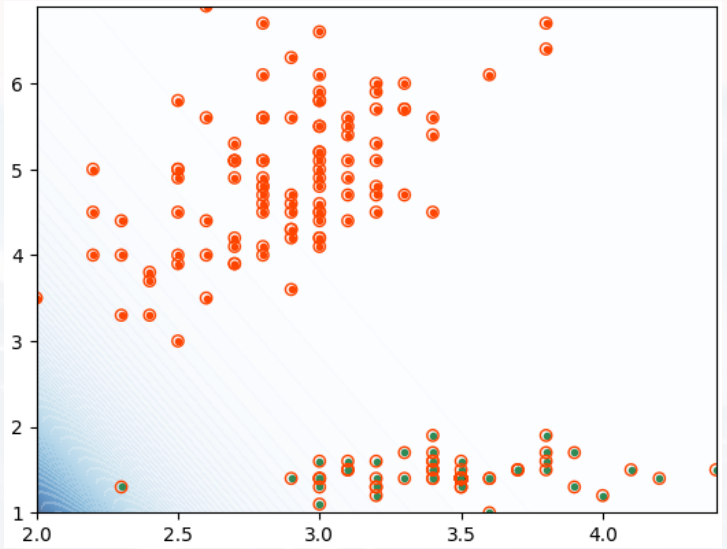




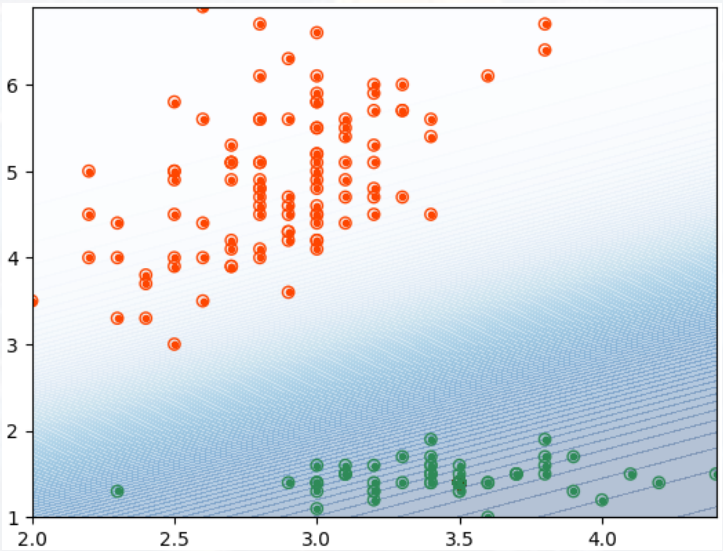
The training process



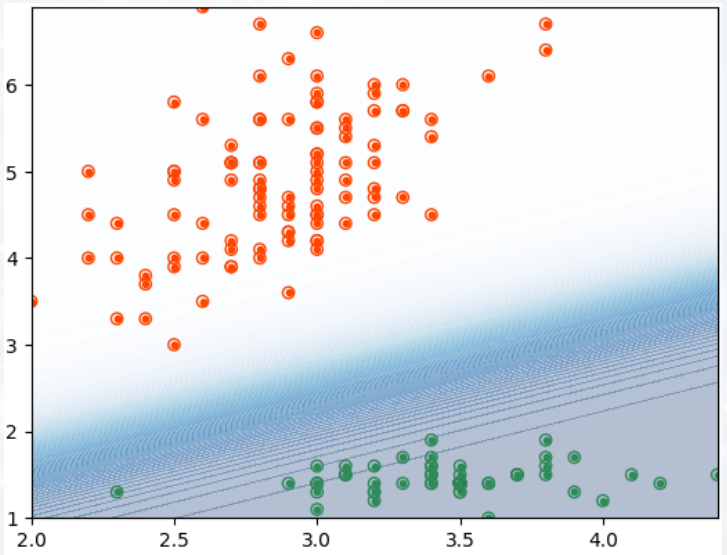
t = 3



t = 5



t = 100



t = 10,000

Thank you for your attention!

